

22/08/2026

740:

```
class Solution {
    public int deleteAndEarn(int[] nums) {
        Map<Integer, Integer> points = new HashMap<>();
        for (int num : nums) {
            points.put(num, points.getDefault(num, 0) + num);
        }
        int maxValue = Arrays.stream(nums).max().getAsInt();
        int[] dp = new int[maxValue + 1];
        dp[0] = 0;
        dp[1] = points.getDefault(1, 0);
        for (int i = 2; i < dp.length; i++) {
            dp[i] = Math.max(dp[i - 2] + points.getDefault(i, 0), dp[i - 1]);
        }
        return dp[maxValue];
    }
}
```

416:

```
class Solution {
    public boolean canPartition(int[] nums) {
        int totalSum = 0;
        for (int num : nums) totalSum += num;
        if (totalSum % 2 != 0) return false;
        int targetSum = totalSum / 2;
        boolean[] dp = new boolean[targetSum + 1];
        dp[0] = true;
        for (int num : nums) {
            for (int currSum = targetSum; currSum >= num; currSum--) {
                dp[currSum] = dp[currSum] || dp[currSum - num];
            }
            if (dp[targetSum]) return true;
        }
    }
}
```

```

    }
}
return dp[targetSum];
}
}

```

[455A - Boredom](#)

```

import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        if (!scanner.hasNextInt()) return;
        int n = scanner.nextInt();
        long[] count = new long[100005];
        long maxVal = 0;
        for (int i = 0; i < n; i++) {
            int val = scanner.nextInt();
            count[val] += val;
            if (val > maxVal) {
                maxVal = val;
            }
        }
        long[] dp = new long[(int) maxVal + 1];
        if (maxVal >= 1) {
            dp[1] = count[1];
        }
        for (int i = 2; i <= maxVal; i++) {
            dp[i] = Math.max(dp[i - 1], dp[i - 2] + count[i]);
        }
        System.out.println(dp[(int) maxVal]);
    }
}

```

